

Heuristics & Simulated Annealing

6DATA005W · Operational Research & Optimisation

Sahdiya Hussain

LECTURE

60 minutes

TUTORIAL

60 minutes

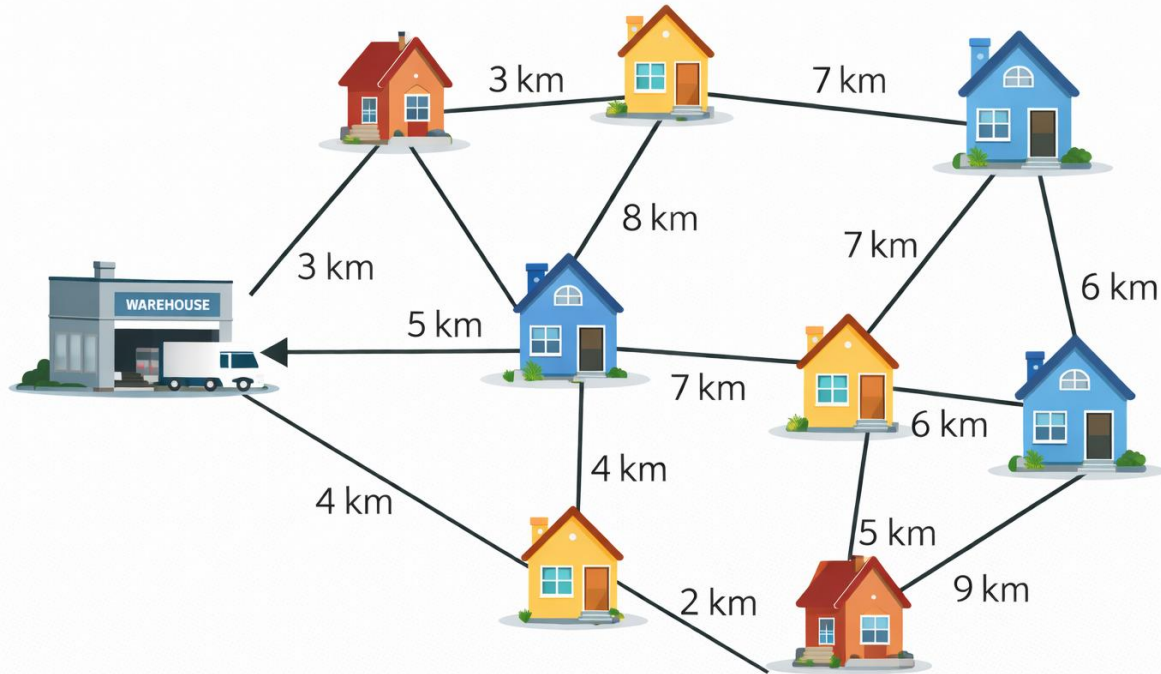
FOCUS

TSP · Heuristics · SA

Why Are Some Problems Hard?

NP-Hardness · The Travelling Salesperson Problem · The Scale Explosion

Find the shortest route (starting and ending at the warehouse)
that visits each house exactly once.



The Travelling Salesperson Problem

"Imagine you're an Amazon driver with 10 stops. You want the shortest route. Simple? Let's count: with 10 cities there are 362,880 possible routes. With 30 cities — 8,400 years of checking, even on the fastest computer."

Formal Definition

Find the shortest tour that visits each city exactly once and returns to the start.

Distance city $i \rightarrow j = c_{ij}$

Classification: NP-Hard

No known fast algorithm for large n

Real-World Versions



Amazon / UPS delivery routes



Uber & ride-share scheduling



Circuit board drilling paths



Telescope observation planning



DNA sequencing assembly



Airline crew scheduling

The Scale Explosion — Why Brute Force Fails

Cities (n)	Possible Tours	Time to Check All	Verdict	NP-Hard means no fast algorithm exists for large n
10	362,880	Instant	✓ Fine	
20	1.2×10^{17}	2.4 milliseconds	⚠ Getting big	
25	6.2×10^{23}	4.2 hours	⚠ Getting bad	
30	8.8×10^{30}	8,400 years	✗ Impossible	
35	2.9×10^{38}	326 billion years	✗ Never	

At n=30: checking every route on the fastest supercomputer would take 8,400 years → We need smarter methods

Understanding NP-Hardness



P — Polynomial Time

Can be SOLVED quickly

A real computer can find the exact answer in polynomial time.

Example: sorting a list, finding shortest path (Dijkstra's algorithm).



NP — Nondeterministic Polynomial

Easy to VERIFY, hard to SOLVE

Given a solution, we can check it quickly — but finding one from scratch is hard on any real machine.

Example: Given a TSP tour, checking its total length is instant.



NP-Hard

At least as hard as all NP problems

TSP falls here. No known polynomial algorithm. May not even be in NP.

No computer — however fast — can solve large TSP exactly in reasonable time.



P = NP ?

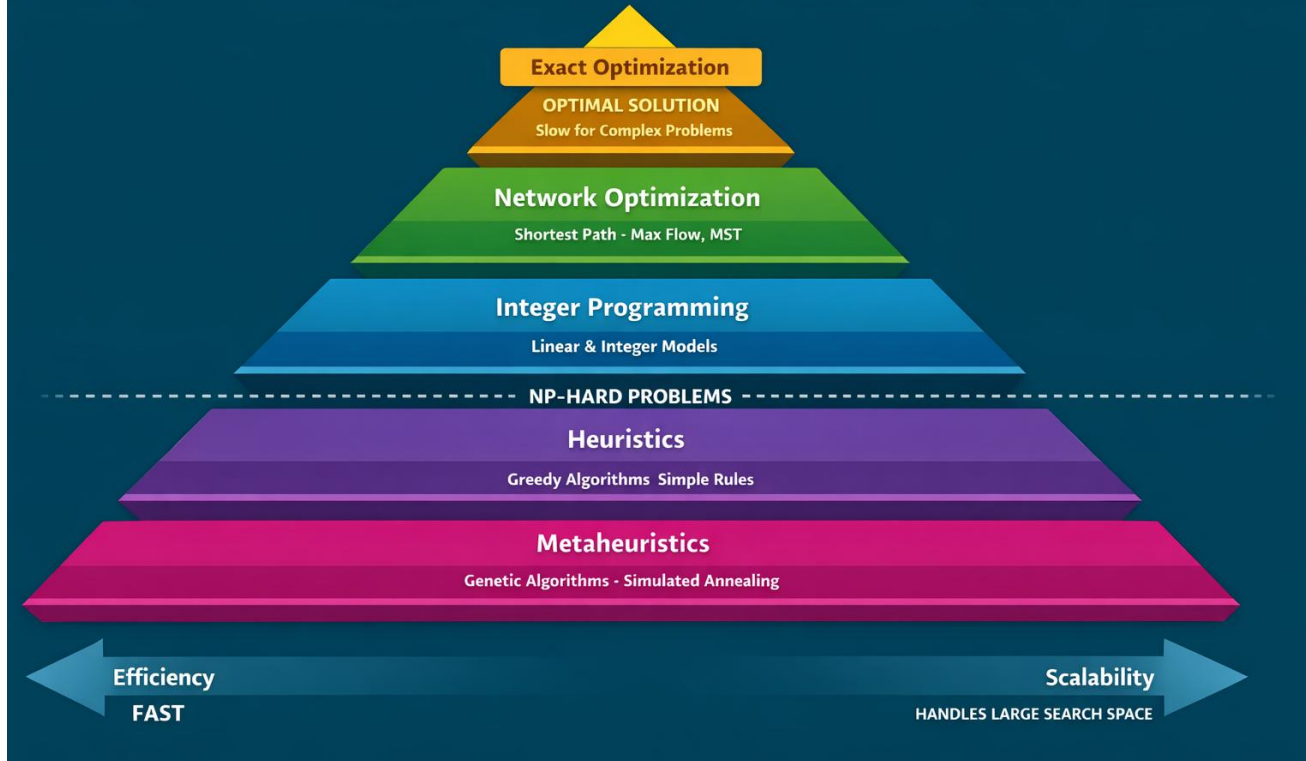
The \$1,000,000 unsolved question

If true, every problem that's easy to verify would also be easy to solve. Most experts believe $P \neq NP$, but nobody has proved it.

Introduction to Heuristics

Definition · Two Types · Nearest Neighbour · Neighbourhoods · 2-opt

Optimization Methods



What is a Heuristic?

"A heuristic is like a recipe that usually makes great food but doesn't guarantee a Michelin star. It's a principled shortcut — we trade the guarantee of a perfect answer for a very good answer, fast. From Greek: heuriskein — to discover or find."

From Greek: heuriskein — to discover or find (by trial and error)

Speed

Finds a good solution in seconds or minutes, not millions of years.

Quality

Usually close to optimal — often within 1–5% of the true best answer.

Trade-off

We give up the guarantee of optimality to gain computational feasibility.

Key Limitation:

Heuristics cannot guarantee finding the global optimum. We often don't know how far the result is from optimal.

Two Families of Heuristics



Construction Heuristics

Builds a solution from SCRATCH one piece at a time.

- Also called 'Greedy' heuristics
- Each step: pick the locally best option
- Fast and simple to implement
- Does NOT look ahead — can make poor long-term choices
- Example: Nearest Neighbour for TSP



Improvement Heuristics

Starts with any complete solution and IMPROVES it through small changes.

- Also called 'Neighbourhood Search'
- Start with a full (possibly random) solution
- Each step: make a small change, check if better
- Can escape greedy traps (local optima)
- Example: Simulated Annealing, Tabu Search

Construction: Nearest Neighbour Heuristic

SAY THIS

"You want to visit 8 museums. At each one, you walk to the nearest museum you haven't visited yet. Fast and decent — but you might end up making a huge detour at the end."

1 Pick any starting city

Often chosen randomly.

2 Go to nearest unvisited city

Check all remaining cities; pick closest.

3 Repeat until all cities visited

Continue greedily at each step.

4 Return to start — close the loop

This may force a long final edge: the greedy trap.

⚠ Greedy trap: good early choices can force very long detours at the end

Typical Quality

Optimal



Nearest Nbr



Random



vs. optimal

Search Space & Neighbourhoods

SAY THIS

"Imagine you're on a foggy hillside trying to reach the highest point. You can only see a few metres around you. You always walk uphill — and eventually stop. But you might be on a small hill, not the tallest mountain."



Search Space S

ALL possible solutions — feasible and infeasible.

For 10-city TSP: 362,880 points in this space.



Neighbourhood $N(s)$

Solutions 'close' to current s — reachable by ONE small change.

The definition of 'close' defines your algorithm.



Local Optimum

Better than ALL its neighbours — but not necessarily the global best.

Local search gets stuck here.



2-opt for TSP

Remove 2 edges, reconnect differently.

If roads cross, uncrossing always shortens the tour — guaranteed improvement.

The Heuristics Landscape

Algorithm	Category	Motto	Key Mechanism
Greedy / Nearest Neighbour	Construction	<i>"Take the best right now"</i>	Build solution step by step
Hill Climbing	Local Search	<i>"Go uphill, stop at top"</i>	Accept only improvements
Simulated Annealing ★	Meta-heuristic	<i>"Shake, then slowly settle"</i>	Occasionally accept worse moves
Tabu Search	Meta-heuristic	<i>"Don't revisit mistakes"</i>	Memory of recent moves; forbid repeats
Genetic Algorithm	Population	<i>"Survival of the fittest"</i>	Evolve a population of solutions
Ant Colony ★ <i>Today's focus</i>	Population	<i>"Follow the pheromone trail"</i>	Stigmergy / pheromone reinforcement

Simulated Annealing

Physical Analogy · 6-Step Algorithm · Acceptance Formula · Cooling Schedule · Pseudocode

Simulated Annealing — The Physical Analogy

"When a blacksmith heats metal to white-hot and slowly cools it, atoms rearrange into the lowest-energy crystal structure — nature's optimum. Cool too fast and atoms get stuck in a brittle, suboptimal shape. SA does the same with solutions: start hot (accept almost anything), cool slowly (get more selective), and hopefully land near the global optimum."

Physical Process

- Heat metal → atoms move freely and randomly
- Slow cooling → atoms find best low-energy positions
- Quench (fast cool) → atoms get stuck, brittle result
- Result: optimal crystal structure

Algorithm Parallel

- High temperature → accept almost any move (explore)
- Gradual cooling → become more selective
- Cool too fast → trapped in local optimum
- Result: near-optimal solution

The SA Algorithm — 6 Steps

1 Initialise

Start with any solution x . Set $T = T_0$ (large). Record x as best.

2 Generate random neighbour y

For TSP: randomly pick two cities and swap their positions.

3 Calculate Δ (delta)

$\Delta = \text{cost}(y) - \text{cost}(x)$. Negative Δ = improvement. Positive Δ = worse.

4 Accept or reject?

If $\Delta \leq 0$: always accept. If $\Delta > 0$: accept with $P = \exp(-\Delta/T)$.

5 Update best solution

If current solution is better than best seen so far, update best.

6 Cool down, then repeat

$T = \alpha \times T$ (e.g. $\alpha = 0.95$). Return to step 2 until stopping condition.

The Acceptance Probability Formula

$$P = \exp(-\Delta / T)$$

Metropolis Criterion · from Statistical Physics (Boltzmann, 1877) · core of Simulated Annealing

Δ (Delta)

How much WORSE the new solution is.
Bigger $\Delta \rightarrow$ lower probability of accepting.
Always positive in this context (worse move).

T (Temperature)

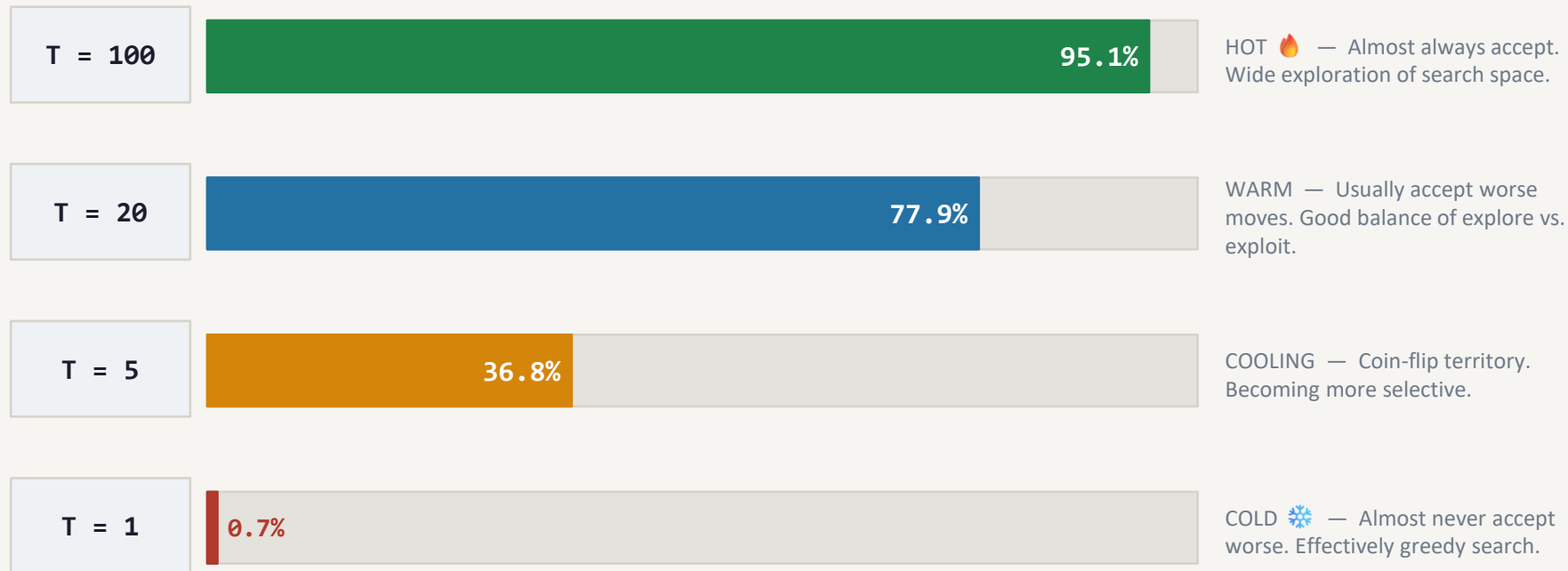
Current temperature.
Large T $\rightarrow$ P close to 1 (accept freely).
Small T $\rightarrow$ P close to 0 (very selective).

P (Probability)

Output: always between 0 and 1.
Generate random $r \in [0,1]$.
If $P > r \rightarrow$ accept. If $P \leq r \rightarrow$ reject.

Temperature Controls Acceptance ($\Delta = 5$, fixed)

As temperature drops, the probability of accepting a worse move falls dramatically:



As $T \rightarrow 0$: $\exp(-\Delta/T) \rightarrow 0 \rightarrow$ SA becomes pure greedy (hill climbing) · As $T \rightarrow \infty$: $P \rightarrow 1 \rightarrow$ SA becomes random walk

The Cooling Schedule — Key Design Decision

Starting Temperature T_0

Hot enough so ~80% of worse moves accepted initially.

Rule of thumb: $T_0 = -\Delta_{avg} / \ln(0.8)$

Ensures exploration, not random walk.



Geometric Cooling (Most Common)

$T = \alpha \times T$ where $\alpha \in [0.85, 0.99]$

$\alpha = 0.99$: slow & high quality

$\alpha = 0.80$: fast & lower quality

Linear Cooling

$T = T - x$ (subtract a fixed amount)

Simpler but less flexible.

Can cool too fast toward the end.



Stopping Condition

Stop when:

- T falls below a threshold, OR
- No improvement for N iterations, OR
- Fixed iteration count reached

Theoretical result: with infinite time and correct geometric cooling, SA converges to global optimum with probability 1 (Geman & Geman 1984)

SA Pseudocode — Walk Through Line by Line

```
// Step 1: Initialise
x ← random_solution()
T ← T0      // e.g. T0 = 100
best ← x

For i = 1 to max_iterations:
    y ← swap_two_cities(x)
    Δ ← cost(y) - cost(x)
    if Δ ≤ 0:
        x ← y      // always accept
    else:
        P ← exp( -Δ / T )
        if random(0,1) < P:
            x ← y    // accept worse
    if cost(x) < cost(best):
        best ← x
    T ← α × T      // cool down
return best
```

Key Points

- ① Start hot — T_0 large enough that ~80% of worse moves are accepted initially
- ② `swap_two_cities` = the neighbourhood function. One random swap per iteration
- ③ $\Delta \leq 0$ means improvement — always accept immediately, no probability needed
- ④ $\exp(-\Delta/T)$ gives a probability — compare with `RAND()` for a yes/no decision
- ⑤ Track 'best' separately — current may worsen; best never gets worse
- ⑥ $\alpha \times T$ is geometric cooling. Typical $\alpha = 0.90 - 0.99$ in practice

TUTORIAL · 60 MINUTES

Hands-On Activities

TSP by Hand · SA Iteration on Paper · Scenic Valley Simulation

Manual TSP — The 8-City Coffee Shop Problem

"Working in pairs — find the shortest tour visiting all 8 cities and returning to A. You have 5 minutes. Start with intuition, then try Nearest Neighbour."

Round 1
3 min

Find any tour by intuition. Write down your route and estimate total distance.

Round 2
3 min

Apply Nearest Neighbour starting from A. Is it better or worse than intuition?

Round 3
2 min

Try swapping any two cities in your tour. Did it improve?

Debrief: What rule were you using in Round 1? Did anyone beat nearest-neighbour? Why might intuition beat NN on small problems?

Teacher note: NN tour $A \rightarrow G \rightarrow F \rightarrow H \rightarrow B \rightarrow C \rightarrow E \rightarrow D \rightarrow A \approx 28.4$ units. Human eye usually finds 24–26.

City Coordinates

City	X	Y
A (Start)	1	5
B	3	8
C	7	9
D	9	5
E	8	7
F	6	2
G	2	3
H	5	6

Each unit = 1 km

One SA Iteration On Paper — Worked Example

Given: Current Tour: 1→3→5→2→6→4→1 Cost = 87 Temperature $T = 20$ $\alpha = 0.95$

Move: Randomly swap cities 3 and 6 → New Tour: 1→6→5→2→3→4→1 New Cost = 94

1 Calculate Δ

$\Delta = 94 - 87 = +7$ → Positive: the new tour is WORSE by 7 units.

2 Is $\Delta \leq 0$?

No — $\Delta = +7$. We cannot auto-accept. Must calculate acceptance probability P .

3 Calculate P

$P = \exp(-7/20) = \exp(-0.35) = 0.705$ → 70.5% chance of accepting.

4 Random number test

$r = 0.62$: $P(0.705) > r(0.62)$ → ACCEPT

$r = 0.81$: $P(0.705) < r(0.81)$ → REJECT

5 Update temperature

$T_{\text{new}} = \alpha \times T = 0.95 \times 20 = 19.0$ (slightly cooler for next iteration)

Student Exercises — Try It Yourself

Exercise 1

Given: Current cost = 102 New cost = 99 $T = 15$ $\alpha = 0.90$ $r = 0.45$

(1) Calculate Δ (2) Is this an improvement? (3) Accept automatically, or calculate P ? (4) New temperature?

Answer 1:

$\Delta = 99 - 102 = -3$ (improvement, $\Delta \leq 0$) $\rightarrow$ Accept automatically, no P needed. New $T = 0.90 \times 15 = 13.5$

Exercise 2 (harder) — Current cost = 75 New cost = 82 $T = 10$ $\alpha = 0.95$ $r = 0.55$

(1) Δ ? (2) $P = \exp(-\Delta/T)$? (3) Accept or Reject? (4) New T ?

Answer 2:

$\Delta = +7$ (worse). $P = \exp(-7/10) = 0.497$. $P(0.497) > r(0.55)$? NO $\rightarrow$ Reject. New $T = 0.95 \times 10 = 9.5$

Scenic Valley SA Simulation

File: natcor_sa_start.xls · 500 iterations · Original tour = 105 → Optimised = 73 (30% saving)

Column	Contents	Formula / Logic
Current Tour	City sequence for this iteration	Copied / updated from previous row
Current Cost	Total tour distance	Sum of distances between consecutive cities
Swap Cities	Two positions to swap	=RANDBETWEEN(1,n) – ensure they differ
New Tour	Tour after the swap	Copy current tour, apply the swap
Δ (Delta)	New – Current cost	=New Cost – Current Cost
Temperature T	Current temperature	= $T_0 \times \alpha^{\text{iteration}}$
Probability P	Acceptance probability	=IF($\Delta < 0$, 1, EXP($-\Delta/T$))
Random r	Random 0–1 number	=RAND()
Accept?	Yes or No decision	=IF($P > r$, "Accept", "Reject")
Next Tour	Tour carried forward	=IF(Accept, New Tour, Current Tour)

Debrief — Discussion Questions

Q1 Run the simulation 3 times. Do you get the same answer?

→ No — SA is stochastic. Results vary each run. That's why we run multiple times and take the best.

Q2 Change α from 0.95 to 0.80. What happens to quality?

→ Cools faster → less exploration → usually finds a worse solution. Cooling schedule design matters.

Q3 What if $T_0 = 0$ from the start? What does SA become?

→ Pure greedy / hill climbing — accepts only improvements. Gets stuck in local optima immediately.

Q4 Original = 105, best = 73. That's a 30% saving. What does this mean for 1,000 Amazon deliveries per day?

→ Significant real-world fuel, time, and cost savings. Heuristics have direct economic impact.

Q5 Can you design a better cooling scheme?

→ Reheat if stuck, adaptive α , multiple restarts from different seeds, non-geometric schedules.

Common Student Misconceptions to Address

✗ SA always finds the optimal solution

✓ SA finds a near-optimal solution. Theoretically it can converge to global optimum with infinite time — in practice it's always an approximation.

✗ SA accepts ALL worse moves at high temperature

✓ It accepts worse moves probabilistically. Even at high T , a catastrophically bad move (huge Δ) still has lower probability. The formula handles this automatically.

✗ Heuristics are just guessing

✓ Heuristics are principled, mathematically-motivated algorithms. SA comes directly from statistical physics. It's engineering, not guessing.

✗ Higher starting temperature is always better

✓ Too high = random walk (wastes time). Too low = greedy (gets stuck immediately). The right T_0 gives ~80% initial acceptance of worse moves.

Session Summary



NP-hard problems (like TSP) cannot be solved exactly at scale — brute force fails beyond ~20 cities



Heuristics trade optimality guarantees for speed — construction heuristics build solutions; improvement heuristics refine them



Simulated Annealing mimics metal cooling: start hot (explore), cool slowly (select), settle near global optimum



Acceptance probability $P = \exp(-\Delta/T)$ controls explore vs. exploit — directly borrowed from Boltzmann statistical physics



Cooling schedule (α , T_0 , stopping condition) is the key design decision — $\alpha = 0.90$ – 0.99 in practice

"Real-world problems are too complex for exact methods — heuristics and SA are the professional's best tool."